Part 2: Evaluation

Figures come from `part2/evaluation_results.csv` (generated by `python -m part2.Evaluation`). The ranking evaluated is the TF-IDF ranking over the AND matches (`search` in `InvertedIndex.py`). BM25 is only an extra. Products without a label count as non-relevant. AP@K is divided by min(K, number of relevant products). AP (no K) is the average precision over the whole ranking, divided by the number of relevant products. The last column of each table is the mean over its queries: the means of the AP@K rows are MAP@K, the mean of AP is the MAP, and the mean of RR is the MRR.

1. Indexing

Code and results: https://github.com/Vagaroto/RIAW-Presentaci-nFinal (folder `Part2/`). The text of each product is the one prepared in Part 1 (title, description, sub-category, brand and product-details values, normalized, without stop words and stemmed; see `Part1/Report_Part1.md`).

Inverted index (`build_index` in `InvertedIndex.py`). For each term it stores its postings, a list of `(pid, tf)` pairs with the number of times the term appears in the product, and its `idf = ln(N / df)`, with N = 28,080 products and df the number of products containing the term.

Queries. The query goes through the same preprocessing as the products. The search is conjunctive (`get_query_index`): a product is returned only if it contains all the query terms, which is the intersection of the postings of those terms.

Test queries. Queries 1 and 2 are the validation queries. We defined five more, chosen from `part2/term_stats.csv`, which lists every term with its df, total tf, idf and tf-idf, so that each query is made of terms a user could type and has a known frequency profile. The AND matches are all the products with every term, before ranking.

#	Query	AND matches	Why it was chosen
3	men polo t-shirt	1,451	short query of popular terms (men: df 14,233, t-shirt: df 12,939) and one mid-frequency term (polo: df 3,055)
4	men round neck half sleeve cotton t-shirt	2,046	long query made only of very common terms (all idf below 1.2): can tf-idf still separate the matches?
5	reebok men t-shirt	72	brand query: the brand field is cut (REEB) but the full name appears in the description
6	men raincoat	23	rare term (raincoat: df 34, idf 6.72) next to a very common one
7	white linen shirt	24	colour plus material: white (df 3,995) is common, linen (df 377) is rarer

Ranking. The ranking of the system is TF-IDF (`tf_idf` and `search`): the score of a product is the sum over the query terms of `tf(t, d) * idf(t)`, and the AND matches are returned in decreasing score order. It has no document length normalization. BM25 (`bm25`, k1 = 1.4, b = 0.25) is implemented only as an extra, to compare with TF-IDF in the discussion, and is not used in any table of this report.

2. Results on the validation queries

Metric	Query 1: women full sleeve sweatshirt cotton	Query 2: men slim jeans blue	Mean (MAP / MRR)
P@5	0.200	0.000	0.100
R@5	0.077	0.000	0.038
AP@5	0.067	0.000	0.033
F1@5	0.111	0.000	0.056
NDCG@5	0.170	0.000	0.085
P@10	0.200	0.000	0.100
R@10	0.154	0.000	0.077
AP@10	0.058	0.000	0.029
F1@10	0.174	0.000	0.087
NDCG@10	0.179	0.000	0.090
P@20	0.150	0.000	0.075
R@20	0.231	0.000	0.115
AP@20	0.056	0.000	0.028
F1@20	0.182	0.000	0.091
NDCG@20	0.195	0.000	0.097
AP	0.109	0.022	0.065
RR	0.333	0.026	0.179

3a. Test queries

The five queries of Part 1 are labeled in `part2/test_labels.csv` (1 relevant, 0 not) for the top 20 results of each, applying fixed criteria to the fields of the original product (`part2/LabelTestQueries.py`):

Query	A product is relevant if
3 `men polo t-shirt`	men's t-shirt whose neck type is Polo Neck
4 `men round neck half sleeve cotton t-shirt`	men's t-shirt with Round Neck, Half Sleeve (the data has a separate Short Sleeve value) and cotton in the fabric
5 `reebok men t-shirt`	men's t-shirt whose brand is Reebok (the brand field is cut to REEB; all 20 name Reebok in the description)
6 `men raincoat`	raincoat for men
7 `white linen shirt`	shirt (not t-shirt) with colour White and linen in the fabric (blends count)

Relevant products found among the 20 judged: query 3: 19, query 4: 9, query 5: 20, query 6: 20, query 7: 16. The judging is ours, based on structured fields, so it is stricter on the attributes than a user reading only the title would be.

Metric	Query 3	Query 4	Query 5	Query 6	Query 7	Mean (MAP / MRR)
P@5	0.800	0.000	1.000	1.000	0.600	0.680
R@5	0.211	0.000	0.250	0.250	0.188	0.180
AP@5	0.543	0.000	1.000	1.000	0.420	0.593
F1@5	0.333	0.000	0.400	0.400	0.286	0.284
NDCG@5	0.661	0.000	1.000	1.000	0.616	0.655
P@10	0.900	0.000	1.000	1.000	0.700	0.720
R@10	0.474	0.000	0.500	0.500	0.438	0.382
AP@10	0.707	0.000	1.000	1.000	0.476	0.637
F1@10	0.621	0.000	0.667	0.667	0.538	0.498
NDCG@10	0.780	0.000	1.000	1.000	0.678	0.692
P@20	0.950	0.450	1.000	1.000	0.800	0.840
R@20	1.000	1.000	1.000	1.000	1.000	1.000
AP@20	0.863	0.294	1.000	1.000	0.726	0.777
F1@20	0.974	0.621	1.000	1.000	0.889	0.897
NDCG@20	0.887	0.521	1.000	1.000	0.871	0.856
AP	0.863	0.294	1.000	1.000	0.726	0.777
RR	0.500	0.083	1.000	1.000	1.000	0.717

Only the top 20 of each query were judged, so the relevant set of a test query is made of the relevant products among those 20. This makes R@20 equal to 1.000 by construction, and R@5 and R@10 are relative to that pool. Recall on the test queries is therefore not comparable with the validation queries.

3b. The metrics

All of them take a ranked list and a set of relevant products. They differ in what they look at:

- **P@K** is the share of relevant products among the first K. It says how clean the top of the list is, but not where inside the K the hits are, and not how many relevant products exist. Query 1 has P@5 = P@10 = 0.200 because the hits are at ranks 3 and 8: going from K=5 to K=10 adds one hit and five more slots.
- **R@K** is the share of all the relevant products found in the first K. It says coverage and can only grow with K, but it ignores order and how many irrelevant products come along. Query 1 reaches 0.231 at K=20, which is 3 of the 13 relevant products.
- **AP@K** averages the precision at each rank where a relevant product appears, so it rewards finding them early. It mixes precision, recall and order in one number. For query 1, P@10 = 0.200 but AP@10 = 0.058: the two hits at ranks 3 and 8 have precision 1/3 and 2/8, divided by min(10, 13).

- **F1@K** is the harmonic mean of P@K and R@K. It gives a single trade-off number and is low when either is low. Here both are low (F1@10 = 0.174 for query 1).
- **MAP** is the mean of AP over queries. It summarises a system with one number and treats every query equally, so one query that fails (query 2) pulls it down (MAP = 0.065 and MAP@10 = 0.029 on the validation queries).
- **MRR** is the mean of 1 / rank of the first relevant product. It only looks at the first hit, which fits a user who wants one good answer. It ignores everything after it. Query 1 has RR = 0.333 (first hit at rank 3) and query 2 has 0.026 (rank 39), so MRR = 0.179.
- **NDCG@K** discounts each hit by log2(rank + 1) and divides by the best possible value. It is the only one that would use graded relevance; here the labels are binary, so it behaves like a smoothed AP with a slower decay (NDCG@10 = 0.179 for query 1).

Analysis of the results. All the values are low, and query 2 has zero for every K. Evidence for why:

- The 40 labeled products rank deep in our lists: for query 1 they sit between ranks 3 and 429 of the 500 matches, and for query 2 the eight labeled products that the AND filter keeps sit between ranks 39 and 210 of 222. Everything above them is unlabeled and counts as non-relevant.
- Reasoning, not proven by the data: some of those unlabeled top results may be relevant, so P@K and NDCG@K probably understate the real quality. The labels look like the top results of another retrieval system, and a metric computed from incomplete judgments favours a system that behaves like that one.
- The RR values and the first-hit ranks agree with the P@K values, which supports that the numbers are consistent.

With only 2 queries and 40 labels, differences of a few hundredths between K values or queries should not be read as real.

Test queries. The five queries show how the metrics separate different behaviours:

- Queries 5 and 6 (a rare term, a brand or a product type) have P@K, AP@K and NDCG@K equal to 1.000: every one of the 20 judged results is relevant, so ranking cannot be judged and only recall grows with K. They are easy queries, and for them the metrics cannot tell a good ranking from a random one.
- Query 3 has P@10 = 0.900 but RR = 0.500 and AP@5 = 0.543: the first result is a V-neck t-shirt, so the first relevant product is at rank 2. P@K barely notices one error, while RR and AP@K, which care about the position, do. This is the case where the metrics disagree.
- Query 4 has P@5 = P@10 = 0.000 and then P@20 = 0.450: the 9 relevant products are at ranks 12 to 20 (RR = 0.083 = 1/12). The top 10 are t-shirts with round neck and the right gender, but they are polyester, or short or full sleeve. A metric at a single K (P@10) says the system failed; P@20, NDCG@20 and AP@20 show it was just late.
- Query 7 has RR = 1.000 but P@5 = 0.600: the first result is relevant, but 2 of the first 5 are not, so MRR hides the problems that P@K shows.

Reading the results together: MRR looks at the first hit only, P@K at the set, and AP@K and NDCG@K at the order. The mean over queries (MAP = 0.777, MAP@10 = 0.637, MRR = 0.717) is dominated by the easy queries 5 and 6.

3c. Limitations of the system and possible fixes

1. **AND is too strict and caps recall.** For query 2, 12 of the 20 labeled products are not in the AND results, and 4 of the 10 relevant ones are among them: one lacks the term `slim`, and three are women's jeans that the labels count as relevant for `men slim jeans blue`. Even a perfect ranking could reach at most R = 6/10 on that query. Fixes: fall back to a ranked OR (a product with most terms) when AND returns few results, or expand the query. The women's jeans are probably a labeling effect (reasoning: the reference retrieval may have matched `men` inside `women`), so the stemmer treating `men` and `women` as different terms is correct for a real user.

2. **TF-IDF has no length normalization, so ties and repetition decide the top.** For `men polo t-shirt`, the first result is a V-neck t-shirt (score 79.2, the next is 26.7) because it repeats the terms in a long description. BM25, kept as an extra, puts only polo-neck t-shirts in its top 3. Fixes: sub-linear tf (1 + log tf), length normalization, or BM25 as the main ranking.

3. **All fields are merged into one text, so a match in the description counts like one in the title.** For `white linen shirt`, the top results match `white` and `linen` only in `description` and `product_details`; the title contains just `shirt`, and 4 of the 20 judged results are not relevant (two are not linen, two are mainly dark blue with white as a second colour). In query 4, the top 10 have no relevant product although all are round-neck men's t-shirts: they are polyester or full sleeve, so `half sleeve` and `cotton` are not satisfied in the field values (reasoning: those words probably match elsewhere in the long merged text). The first result of query 3 is a V-neck t-shirt whose description is a list of search terms that includes `Polo Collar`, `Round Neck` and `V neck`, which is why a long, keyword-stuffed text wins. In query 1, 7 of the 20 labeled products are irrelevant (four t-shirts and three sweatshirts) although all pass the

AND filter. Fixes: keep a separate index per field and combine the scores with weights (BM25F style) or boost the title. Weights should be fitted on more judgments than the current 40, otherwise they inflate the evaluation (decision already taken in Part 1).

4. **Numeric fields are not used in ranking.** Rating, price, discount and stock are kept in each product but ignored, so products with the same text tie (identical titles with colour variants get the same score) and out-of-stock products can be returned. Fixes: use them as tie-breakers, as a filter for `out_of_stock`, or as a small additional ranking signal.
5. **Query formulation and preprocessing.** The stop-word list removes `no` and `not`, so negations are lost. There is no synonym handling (`tee` vs `tshirt`) and no spelling correction, and a phrase such as `full sleeve` is two independent terms because the index has no positions. Fixes: query expansion with synonyms, a spelling step, and a positional index for phrases.
6. **Indexing strategy.** The index is rebuilt from the 28,080 products on every run and is not saved. Fix: persist it (or the processed products) to disk.
7. **Evaluation setup.** 2 labeled queries with 20 products each, and unjudged products counted as non-relevant, make the metrics noisy and biased (see 3b). Fixes: judge the top results of every system (pooling) and use more queries.